



INDEPENDENT SECURITY REVIEW

September 20, 2026

LI.FI

EXPLODE Opcode (v1.0.0)

Independent Security Review

SECURITY RESEARCHER

Sujith Somraaj (somraajsujith@gmail.com)

REPORT PREPARED BY

Sujith Somraaj

Table of Contents

1	About Researcher	1
2	Disclaimer	1
3	Scope	1
4	Risk classification	1
4.1	Impact	2
4.2	Likelihood	2
4.3	Action required for severity levels	2
5	Executive Summary	2
6	Findings	4
6.1	Low Risk	4
6.1.1	Malformed dynamic tuple offsets can propagate non-canonical ABI into downstream calls .	4
6.1.2	The default deployment path silently retains a VM that rejects EXPLODE	4
6.2	Informational	5
6.2.1	EXPLODE's documented worst-case complexity is overstated	5
6.2.2	EXPLODE documentation incorrectly states that inter-tail bytes are discarded	5
6.2.3	MemoryUtils.slice may panic instead of reverting with OutOfBounds	6

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

ExplodeLib	src/Explode.sol (v1.0.0, new)
VirtualMachine	src/VirtualMachine.sol (v1.1.0)
CommandPacking	src/CommandPacking.sol (v1.1.0)
MemoryUtils	src/MemoryUtils.sol (v1.1.0)
VmConstants	src/VmConstants.sol (v1.1.0)
VmErrors	src/VmErrors.sol (v1.1.0)
DataModel	src/DataModel.sol (untagged struct change)

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

Rating	Definition
High	leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	global losses <10% or losses to only a subset of users, but still unacceptable.
Low	losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

Rating	Definition
High	almost certain to happen, easy to perform, or not easy but highly incentivized
Medium	only conditionally possible or incentivized, but still relatively likely
Low	requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Severity	Required action
Critical	Must fix as soon as possible (if already deployed)
High	Must fix (before deployment if not already deployed)
Medium	Should fix
Low	Could fix

5 Executive Summary

Over the course of 8 hours in total, LI.FI engaged with the **researcher** to audit the contracts described in section 3 of this document ("scope").

This is a differential review, and hence the scope is limited only to the changes introduced in the audit commit.

In this period of time a total of **5 issues** were found.

Project Summary

Project Name	EXPLODE Opcode (v1.0.0)
Project Type	Solidity Virtual Machine
Repository	lifinance/Yggdrasil-VirtualMachine
Commit	f763689
Audit Timeline	September 11, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	2
Gas Optimizations	0
Informational	3
Total Issues	5

6 Findings

6.1 Low Risk

6.1.1 Malformed dynamic tuple offsets can propagate non-canonical ABI into downstream calls

CONTEXT: [Explode.sol#L37-L44](#)

DESCRIPTION: `ExplodeLib.execute` validates that dynamic offsets are aligned, in bounds, and strictly increasing, but it defines each dynamic register's size exclusively from the next dynamic offset (or the end of the source). It does not establish that the resulting slice is the canonical encoding of the field it represents.

An external call target can return a tuple block whose offsets satisfy every enforced shape check while disagreeing with a dynamic field's embedded framing:

- Moving the next offset outward makes the preceding register contain attacker-chosen surplus bytes beyond its declared logical payload.
- Moving it inward can make a register declare more payload than it physically contains, causing the re-encoded field to overlap subsequent field data.

Normal Solidity ABI decoding remains bounded, and the behavior does not by itself permit an out-of-bounds memory read or unauthorized transfer. The security impact is conditional on a downstream consumer hashing, signing, byte-comparing, or manually slicing the raw calldata rather than decoding it. Such a consumer can observe or act on attacker-controlled non-canonical bytes that are not part of the intended logical values.

RECOMMENDATION: Do not use registers produced from untrusted `CALL` returndata by `EXPLODE` in raw-byte hashing, signing, equality, or manual-offset authorization logic without type-aware canonicalization.

If canonical byte identity must become a VM guarantee, extend the command format with enough per-destination ABI type information to validate or reconstruct each dynamic field. A generic length-word check is insufficient because dynamic arrays use element counts and nested dynamic tuples do not necessarily begin with a byte length.

LI.FI: Accepted as designed. Behaviour unchanged and documented in [859780a](#) and [b90aab0](#).

RESEARCHER: Verified fix.

6.1.2 The default deployment path silently retains a VM that rejects EXPLODE

CONTEXT: [DeployAll.s.sol#L12-L21](#)

DESCRIPTION: The branch enables the `EXPLODE` opcode in `VirtualMachine`, but the default deployment path continues to use the salts of the already-deployed VM and proxy factory. On a chain where those deterministic addresses contain code, both deployment helpers return the existing address without verifying its bytecode or configuration.

As a result, running `DeployAll.run()` on an already-supported chain can complete and log deployment addresses while retaining the old VM, where opcode 2 still reverts with `VmErrors.Disallowed()`. It also retains the old proxy factory, whose `vmContract` is immutable. Existing `MinimalProxy` instances likewise store an immutable `vmAddress`, so they cannot be redirected to an `EXPLODE`-enabled VM.

If the compiler or backend begins emitting `EXPLODE` commands after such an apparent deployment, every affected program reverts. This causes a systemic availability failure for `EXPLODE`-dependent flows. Transactions revert atomically, so the issue does not by itself create partial execution or direct loss of funds.

RECOMMENDATION: Treat this opcode addition as a versioned deployment:

- Assign new VM and proxy-factory salts on chains where the current deterministic addresses are occupied.
- Make the idempotency guards verify the expected VM capability or deployed bytecode, and verify that an existing factory's `vmContract` equals the intended VM address. Revert on a mismatch instead of treating any code as a successful deployment.
- Add a post-deployment smoke test that executes a minimal valid `EXPLODE` command through the returned VM and through a proxy created by the returned factory.

- Keep compiler emission and backend exposure of EXPLODE-dependent operations disabled until the new addresses are deployed and configured on every supported chain.
- Document the migration implications for existing immutable user proxies and approvals.

LI.FI: Fixed in [2a26eff](#).

RESEARCHER: Verified fix.

6.2 Informational

6.2.1 EXPLODE's documented worst-case complexity is overstated

CONTEXT: [Explode.sol#L72-L82](#)

DESCRIPTION: The EXPLODE cost note describes dynamic-tail resolution as $O(\text{destCount}^2)$ and states that a 26-destination command may perform 325 inner-loop iterations.

This does not match the implementation. For each dynamic destination, `ExplodeLib._resolveDynamicEnd` scans forward only until it encounters the next dynamic destination and then stops. The portions scanned for successive dynamic destinations therefore do not overlap. Across the complete command, the inner loop performs at most $\text{destCount} - 1$ destination checks. With the current limit of 26 destinations, the maximum is 25 checks, including for an all-dynamic command.

The discrepancy does not create an execution vulnerability or invalidate the gas benchmark, but it gives auditors and integrators an inaccurate model of the opcode's resource consumption.

RECOMMENDATION: Update the cost note to describe destination scanning as linear in `destCount`, with at most 25 aggregate inner-loop checks. Account for memory allocation and copying separately, as their cost depends on the total number of bytes copied into destination registers.

LI.FI: Fixed in [35eeaed](#) and [09321ce](#).

RESEARCHER: Verified fix.

6.2.2 EXPLODE documentation incorrectly states that inter-tail bytes are discarded

CONTEXT: [Explode.sol#L44](#)

DESCRIPTION: The EXPLODE documentation states that bytes in a gap between one dynamic tail's end and the next are discarded. This is inconsistent with both the implementation and the documented non-canonical length behavior.

For a dynamic destination, the implementation defines the slice end as the next dynamic destination's offset, or the end of the source for the final dynamic destination. Consequently, every byte between two successive dynamic offsets is copied into the earlier destination. If the field's embedded ABI length describes a shorter logical payload, the extra bytes are retained as surplus data in that destination rather than discarded.

Only bytes between the end of the tuple head ($\text{destCount} * 32$) and the first dynamic offset are unassigned and discarded. The current wording may cause downstream consumers to incorrectly assume that surplus bytes cannot propagate through EXPLODE and subsequent ABI re-encoding.

RECOMMENDATION: Clarify the documentation as follows:

- Bytes between the tuple head and the first dynamic offset are discarded.
- Every interval from one dynamic offset to the next is copied in full into the earlier dynamic destination.
- EXPLODE does not use an embedded ABI length word to shorten a dynamic slice, so bytes beyond a field's declared logical payload remain present and may survive re-encoding.

LI.FI: Fixed in [4be6b19](#) and [7ba88e3](#).

RESEARCHER: Verified fix.

6.2.3 MemoryUtils.slice may panic instead of reverting with OutOfBounds

CONTEXT: [MemoryUtils.sol#L9](#)

DESCRIPTION: `MemoryUtils.slice` checks whether a requested range exceeds the input by evaluating `start + length > data.length`. Because Solidity performs checked arithmetic, `start + length` reverts with `Panic(0x11)` when the addition overflows. In that case, the function does not return the documented `VmErrors.OutOfBounds()` error.

The condition is not exploitable through the current EXPLODE implementation. Static slices use bounded head offsets, while dynamic slices pass a validated `start` and `end - start`, making the sum equal to a validated `end` within the source. Nevertheless, `MemoryUtils.slice` is a generic internal utility, and a future caller could expose the inconsistent revert behavior or break code that relies on the VM's declared error taxonomy.

RECOMMENDATION: Use an overflow-safe bounds check that validates `start` before subtracting it from the input length:

```
if (start > data.length || length > data.length - start) {
    revert VmErrors.OutOfBounds();
}
```

Add a unit test using `start = type(uint256).max` and a non-zero length to confirm that the function returns `VmErrors.OutOfBounds()` rather than a Solidity panic.

LI.FI: Fixed in [bd73168](#) and [2f45ff7](#).

RESEARCHER: Verified fix.